

Lab 1: Getting Started

ECE 455 / ECE 555, Fall 2026

Assigned	Wednesday, August 19
Due	Friday, September 4, 11:59 p.m. , through Canvas

Objective

- To get set up the programming platform
- To write the code that initializes the Port F
- To write subroutines that read the state of the switches and turn on/off the LEDs connected at Port F
- To demonstrate the use of bit-specific addressing
- To interface one external LED and justify the resistor value you chose

An Embedded System is a combination of Hardware (electronic, perhaps mechanical) and computer software with appropriate functions crafted to perform a specific task. The fundamental learning in programming in Embedded Systems has two aspects. First, to understand how the software is written can affect physical things, and second, to understand what happens down inside the hardware. This assignment is based on Embedded C, for its reliability, scalability, and portability.

Teamwork

Laboratory work is done in teams of two, formed in the first week and kept for the whole semester. Both members must be able to explain every part of the submission to the TA. Every lab in this course is demonstrated in person to the TA during TA office hours. Bring your board, your circuit, and your results. The TA checks the operation, verifies that it meets the assigned functionality **for each individual student**. Both team members demonstrate, and either one may be asked to explain any part of the submission. TA office hours are posted on Canvas. Plan your demonstration in advance rather than in the last session before the due date.

Preparation

- A Tiva TM4C123 LaunchPad and a PC with Keil μ Vision installed
- A breadboard and a few connecting wires
- 1 LED (any color) and an assortment of resistors (you will calculate which one to use)

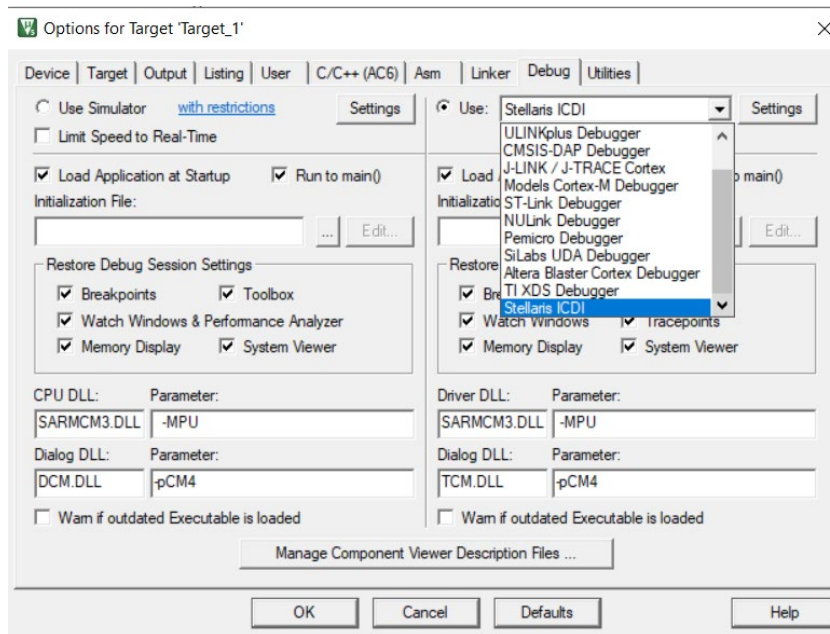
Part 1. Toolchain (6 points)

The first objective of the lab is to install the Keil μ Vision IDE software to write source code, edit the code, debug, and program the microcontroller. You need a Laptop/PC that has the Keil

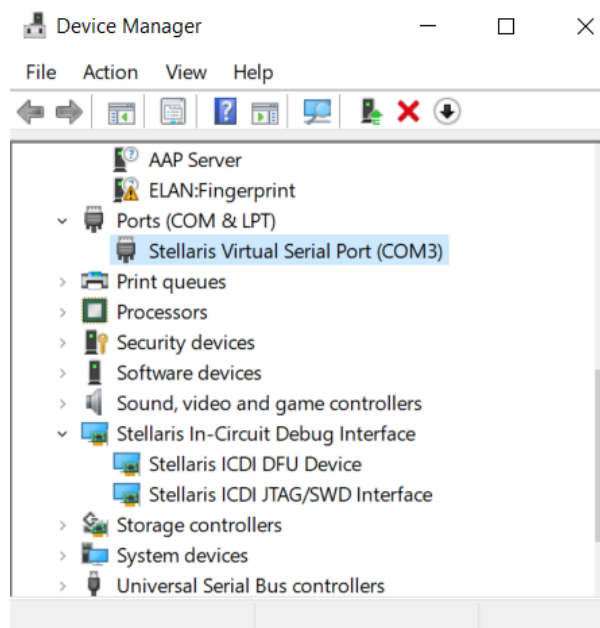
interface and the driver (Stellaris ICDI) for the board TM4C123. For this first part, you need to submit the following two screenshots in your report file.

The Keil μ Vision IDE and Device Driver Installation Guide for the TM4C123 board is posted on Canvas with this lab, as Toolchain_Install_Guide_Fall2026.pdf. Its troubleshooting section covers the problems that come up most often.

- **Keil μ Vision IDE:** Check that menu Project -> Options for Target -> Debug has the Stellaris ICDI driver set.



- Take screenshots of the Stellaris driver installation and of the COM port number of your microcontroller in Device Manager.



Part 2. Port F Initialization (6 points)

The Tiva C TM4C123 microcontroller LaunchPad evaluation board comes with two user switches (connected at **PF0** and **PF4**) and Red (**PF1**), Blue (**PF2**) and Green (**PF3**) LEDs to kick start our application. The other pins can be accessed via the connectors labeled J1 to J4. Before you jump to do programming it is expected that you get yourself familiar with **Figure 4.5** (the LaunchPad board) and **Figure 4.6** (the switch and LED interfaces) in the textbook *Volume 1, Introduction to ARM Cortex-M Microcontrollers* by Jonathan Valvano, PDF pages 182 and 183.

In this part, we expect you to write the PortF_Init subroutine (at the space given in the project directory) that initializes the Port F pins as input/output as per the connection of the switches and LEDs. Generally, the port initialization routine is a seven step procedure that sets certain specific registers. Remember, access to the port register is **memory-mapped I/O**. You will be accessing the port as if you are accessing a memory address, that simple.

The following things are expected as part of the submission in the code:

- Your subroutine PortF_Init has all the necessary steps.
- All your initialization steps should be **friendly** programmed, that is, we expect you to use |= or &= operators, so that a programming statement does not alter the previously specified values.

Note: Until we further specify, all programming statements in your code should be friendly programmed.

Part 3. LED and Switch Subroutines (78 points)

This part has three sub-parts.

3A. Functions without Bit-Specific Addressing (22 points)

You will create a total of 8 small subroutines as follows:

1. Toggle the Red LED (toggle means: LED on, delay, LED off, delay, and repeat)
2. Toggle the Blue LED
3. Toggle the Green LED
4. Turn on Red LED
5. Turn on Blue LED
6. Turn off ALL LEDs
7. Return the state of the pushbuttons
8. A delay function that has a delay in the range of **250 ms to 500 ms**

3B. Combined Behavior (34 points)

Use the subroutines created in the previous part to write code with the following behavior:

- When no switch is pressed: toggle the Red LED, others off.

- **SW1 only:** Green LED on, others off
- **SW2 only:** Blue LED on, others off
- **Both:** The behavior should look like: All LEDs off, then all LEDs on, then Green off with Red and Blue on, then Green on with Red on and Blue off, then start over again. Between each step there should be a delay that is visible to the eye. Further, the code should check the pushbutton states **before each transition**, so that it can exit the pattern early rather than waiting for the entire routine to complete.

3C. Using Bit-Specific Addressing (22 points)

Using bit-specific addressing, create two separate functions to turn on and off the Red (**PF1**) and Green (**PF3**) LEDs. The code should behave as follows:

- **SW1 only:** The LEDs should blink to simulate counting from 0 to 3 in binary, i.e. 00, 01, 10, 11
- **SW2 only:** The LEDs should blink to simulate counting from 3 to 0 in binary, i.e. 11, 10, 01, 00
- **Other cases:** Alternate between 01 and 10

Note: The MSB is the Green LED and the LSB is the Red LED.

Part 4. One External LED (10 points)

Connect one external LED to a general purpose pin that has nothing else attached to it, using positive logic (pin high = LED on) with a series current-limiting resistor. Every Port F pin broken out on this board is already taken: PF1, PF2 and PF3 drive the on-board red, blue and green LEDs, and PF0 and PF4 read the two on-board switches. Use a pin from another port instead, for example PA5 (J1 pin 8) or PB3 (J4 pin 3).

Submit the following:

- **A hand-drawn circuit diagram.** We do not expect you to use any software, however, the drawing should be in compliance with basic Electrical Engineering practice. Check the symbols for the LED, resistor, and ground, and mark the direction of the current. Drawing wire connections without a straightedge, or that do not look straight, will not serve the purpose.
- **An actual photograph of your prototype. R1**
- **Your resistor calculation. R2** State the LED forward voltage V_f you assumed and where you got it, state the current I you targeted, and show:

$$R = (V_{OH} - V_f) / I$$

Then choose the standard resistor value you actually used and state the resulting current through the LED. Justify that your current is within what the pin can supply. The TM4C123 GPIO pins default to 2 mA drive and can be configured for 4 mA or 8 mA (GPIO_PORTF_DR2R_R, DR4R, DR8R).

Report clarity and code quality are not a separate line item; they are graded inside each part. Poorly structured code, or a report in which a required item cannot be found, loses points in the part it belongs to.

Report

Submit Report1.pdf inside a single .zip archive together with **one** Keil μ Vision project folder containing all code.

- The report must display the names of both team members.
- Items marked R1 and R2 appear only in the report. You are free to add information that helps the TA grade; we do not give extra credit for background theory.
- Code must be commented, highlighting any design choices or calculations you made.
- Code must be properly structured and make sense. Poorly structured code, though working, may severely affect your score.
- In embedded systems it is preferred that your code be as simple as possible, so that the translation into binary is shorter and more portable. Writing unnecessary compiler-specific directives is not a good idea.

AI Use Statement

Generative AI tools are permitted on this lab, but their use must be disclosed. End your report with a short AI Use Statement naming the tool, what you used it for, and what it contributed. If you used no AI tool, write "No generative AI tools were used."

Late Work

Submissions are due at 11:59 p.m. on the stated date. **Late submissions are not accepted.** If you must miss a submission date, inform the instructor in advance, or submit a legitimate, documented reason for the absence. Failure to do so results in a grade of zero for this assignment. See the syllabus for the full policy.